

Memoria del Proyecto: SprintForge

Realizado por:

Jaime García de las Heras y Erika Macarro García.

Junio 2026

1. Introducción

El objetivo de esta práctica es el desarrollo de un sistema de alquiler de viviendas, inspirado en plataformas como Airbnb. El sistema permite la interacción entre dos tipos principales de usuarios, **inquilinos** y **propietarios**, facilitando la búsqueda, reserva y gestión de alojamientos de forma segura, estructurada y controlada.

El proyecto, denominado **SprintForge**, se ha desarrollado como un trabajo en grupo formado por dos integrantes, aplicando los principios fundamentales de la Ingeniería del Software con un énfasis en el uso de un proceso de desarrollo bien definido.

Desde el punto de vista tecnológico, el sistema ha sido implementado utilizando **Java** como lenguaje principal, apoyándose en el framework **Spring Boot** para el desarrollo de la lógica de negocio y la capa web. La interfaz de usuario se ha desarrollado mediante **HTML y CSS**, utilizando plantillas gestionadas por **Thymeleaf**, lo que permite una correcta separación entre la lógica de presentación y la lógica de negocio. La persistencia de datos se gestiona mediante **Spring Data JPA**, utilizando una base de datos embebida **Apache Derby**.

El proyecto se ha construido y gestionado mediante **Maven**, lo que ha permitido automatizar el proceso de compilación, ejecución de pruebas y generación de informes. Para la verificación y validación del sistema se han implementado **pruebas automatizadas con JUnit**, integradas en el ciclo de construcción mediante el plugin **Surefire**, complementadas con el uso de **JaCoCo** para el análisis de la cobertura de código y **SonarCloud** para la evaluación de la calidad y mantenibilidad del software.

A nivel funcional, el sistema implementa la totalidad de los requisitos especificados en el enunciado de la práctica, cubriendo tanto los escenarios normales de uso como las situaciones excepcionales derivadas de la confirmación o denegación de solicitudes de reserva, incluyendo la gestión de pagos y devoluciones. Cada funcionalidad ha sido diseñada de forma trazable desde los requisitos hasta el código y las pruebas, garantizando así la coherencia y corrección del sistema.

2. Metodología de desarrollo

Para la organización y desarrollo del **proyecto SprintForge** se ha adoptado una metodología ágil, concretamente Scrum, debido a su enfoque iterativo e incremental y a su adecuación para proyectos software desarrollados en equipo.

Scrum permite dividir el desarrollo en ciclos cortos de trabajo, denominados **sprints**, facilitando la **planificación progresiva**, la adaptación a cambios y la validación continua de las funcionalidades implementadas. Este enfoque ha resultado especialmente adecuado para un proyecto académico con requisitos bien definidos, pero con posibilidad de ajustes durante el desarrollo.

2.1 Organización del trabajo y herramientas

El proyecto se ha desarrollado utilizando **GitHub** como plataforma central para la gestión del código fuente, el control de versiones y la coordinación del trabajo. Esta herramienta ha permitido:

- Mantener un historial completo y trazable de los cambios realizados.
- Facilitar el trabajo colaborativo mediante ramas y fusiones de código.
- Centralizar la documentación y el seguimiento del proyecto.

La construcción del proyecto, la gestión de dependencias y la ejecución de pruebas se han automatizado mediante **Maven**, lo que ha contribuido a mejorar la reproducibilidad y consistencia del proceso de desarrollo.

2.2 Roles dentro del equipo Scrum

Siguiendo las directrices de la metodología Scrum, se han definido los siguientes roles dentro del equipo:

- **Product Owner:** responsable de interpretar el enunciado del proyecto, definir y priorizar los requisitos funcionales del sistema y asegurar que el producto desarrollado cumple los objetivos establecidos.
- **Scrum Master:** encargado de facilitar la coordinación del equipo, supervisar el correcto desarrollo de los sprints y velar por la aplicación de las prácticas ágiles durante el proyecto.
- **Equipo de desarrollo:** responsables de la implementación de las funcionalidades, la realización de pruebas, la corrección de errores y la mejora continua de la calidad del sistema.

La asignación de roles ha permitido una mejor organización del trabajo y una distribución equilibrada de las responsabilidades dentro del equipo.

2.3 Desarrollo iterativo e incremental

El desarrollo del proyecto se ha estructurado en **cuatro sprints**, cada uno con una duración exacta de **dos semanas**. En cada sprint se definieron objetivos claros y alcanzables, priorizando aquellas funcionalidades que permitían construir progresivamente un sistema funcional y coherente.

Este enfoque iterativo ha permitido:

- Validar de forma temprana las funcionalidades desarrolladas.
- Detectar y corregir errores de manera progresiva.
- Reducir riesgos asociados a la integración tardía de componentes.

2.4 Justificación de la metodología empleada

La elección de Scrum como metodología de desarrollo se justifica por las siguientes razones:

- Facilita la división del proyecto en tareas manejables.
- Permite una planificación flexible y adaptativa.
- Favorece la colaboración y la responsabilidad compartida.
- Se ajusta al desarrollo incremental.

2.5 Eventos Scrum

Aunque el proyecto se desarrolló en un entorno académico, se han seguido de forma adaptada los principales **eventos de Scrum**:

- **Planificación del sprint:** al inicio de cada sprint se establecieron los objetivos y se seleccionaron las tareas a realizar, teniendo en cuenta el estado actual del proyecto y el tiempo disponible.
- **Seguimiento del sprint:** durante el desarrollo se realizó un seguimiento continuo del progreso, detectando posibles bloqueos o dificultades técnicas y ajustando el trabajo cuando fue necesario.
- **Revisión del sprint:** al finalizar cada sprint se revisaron las funcionalidades implementadas, comprobando su correcto funcionamiento antes de continuar con el siguiente ciclo de desarrollo.
- **Retrospectiva:** de forma informal, el equipo analizó los problemas encontrados y las mejoras posibles.

Esta adaptación de los eventos Scrum permitió mejorar la organización y la eficiencia del equipo sin introducir una sobrecarga innecesaria de gestión.

2.6 Gestión de cambios y evolución del sistema

Uno de los principios fundamentales de las metodologías ágiles es la capacidad de adaptación al cambio. Durante el desarrollo del proyecto se produjeron ajustes y mejoras sobre la planificación inicial, motivados por:

- Corrección de errores detectados en fases tempranas.
- Mejora de funcionalidades ya implementadas.
- Ajustes derivados de la integración entre módulos.

Estos cambios se gestionaron de forma controlada gracias al uso del sistema de control de versiones y a la estructura iterativa del desarrollo, evitando impactos negativos en la estabilidad del sistema.

2.7 Relación de la metodología con la calidad del software

La metodología empleada ha contribuido directamente a mejorar la calidad del software desarrollado, ya que:

- Favorece la detección temprana de errores.
- Facilita la integración progresiva de funcionalidades.
- Permite validar continuamente el cumplimiento de los requisitos.
- Reduce la complejidad del mantenimiento posterior.

Este enfoque está alineado con los principios de calidad y mantenibilidad del software, sentando una base sólida para las fases de pruebas y mantenimiento del sistema.

3. Planificación del trabajo por sprints

El desarrollo del proyecto **SprintForge** se ha organizado siguiendo un enfoque iterativo e **incremental**, estructurando el trabajo en una serie de **sprints bisemanales**. Esta planificación ha permitido avanzar progresivamente en la implementación del sistema, integrando funcionalidades de forma ordenada y controlada.

Cabe destacar que, aunque la mayor parte del trabajo se ha gestionado y versionado mediante **GitHub**, no todas las actividades realizadas durante los sprints ni los mismos sprints en algún caso quedan reflejadas explícitamente en el repositorio, especialmente aquellas relacionadas con tareas de análisis, diseño o revisión de funcionalidades. No obstante, dichas actividades han sido fundamentales para el correcto desarrollo del proyecto y han seguido las prácticas propias de la **metodología Scrum**.

Sprint 1: Cimientos y Usuarios

Duración: 2 semanas, del 15 de abril al 28 de abril.

Objetivo: Preparar el entorno de desarrollo, establecer la base del sistema e implementar los mecanismos básicos de acceso al sistema.

Tareas realizadas:

- Creación y configuración del repositorio del proyecto.
- Definición de la estructura inicial del proyecto.
- Configuración del sistema de construcción mediante Maven y el archivo pom.xml con dependencias.
- Configuración de la base de datos embebida H2 y verificación de la conexión.
- Implementación del registro de usuarios.
- Gestión de autenticación y cierre de sesión diferenciando roles (inquilino y propietario).
- Creación de las clases principales relacionadas con los usuarios y desarrollo de las vistas asociadas. Este sprint permitió establecer una base técnica sólida y el acceso controlado a las funcionalidades.

Sprint 2: Catálogo y Búsqueda

Duración: 2 semanas, del 29 de abril al 12 de mayo.

Objetivo: Implementar los mecanismos básicos de acceso al sistema.

Tareas realizadas:

- Alta y gestión de inmuebles por parte de los propietarios. Gestión de autenticación y cierre de sesión.
- Implementación de la búsqueda de alojamientos y filtros básicos.
- Desarrollo de las vistas de resultados de búsqueda.
- Implementación de disponibilidad de inmuebles.
- Integración inicial entre usuarios e inmuebles. Este sprint permitió disponer de un sistema funcional para la consulta y gestión de alojamientos.

Sprint 3: Núcleo de Reservas y Pagos

Duración: 2 semanas, del 13 de mayo al 26 de mayo.

Objetivo: Implementar el núcleo funcional del sistema relacionado con las reservas.

Tareas realizadas:

- Implementación de la reserva inmediata de inmuebles.
- Gestión de solicitudes de reserva.
- Integración del sistema de pago simulado (pasarela de pago). Este sprint completó el flujo principal de uso del sistema desde la selección del inmueble hasta el pago.

Este sprint permitió disponer de un sistema funcional para la consulta y gestión de alojamientos.

Sprint 4: Refinamiento y Cierre

Duración: 2 semanas, 27 de mayo al 9 de junio. |

Objetivo: Añadir funcionalidades avanzadas, asegurar la calidad del sistema y preparar la entrega final.

Tareas realizadas:

- Implementación del sistema de notificaciones a inquilinos y propietarios.
- Gestión de confirmaciones, denegaciones y devoluciones.
- Aplicación de filtros avanzados en la búsqueda.
- Gestión de la lista de deseos (favoritos).
- Implementación y ampliación de pruebas automatizadas con JUnit.
- Ejecución de pruebas mediante Maven Surefire y análisis de cobertura de código con JaCoCo.
- Evaluación de la calidad del software mediante SonarQube y revisión final de la documentación.

Este sprint permitió validar el sistema de forma global y garantizar que el producto final cumple los requisitos establecidos.

4. Requisitos del sistema y trazabilidad

En este apartado se describen los **requisitos del sistema**, derivados directamente del enunciado de la práctica, así como su relación con el diseño y la implementación del sistema.

El objetivo es garantizar la **trazabilidad** entre los requisitos planteados, los casos de uso definidos y la solución software desarrollada.

4.1 Requisitos funcionales

Los requisitos funcionales del sistema se corresponden estrictamente con los descritos en el enunciado de la práctica y han sido modelados mediante un **diagrama de casos de uso**, que representa las interacciones entre los actores y el sistema.

A continuación, se describen los requisitos funcionales identificados.

RF1. Registro y autenticación de usuarios

El sistema permite el **registro y autenticación** de usuarios, diferenciando entre **inquilinos y propietarios**.

Ambos tipos de usuario deben estar registrados y autenticados para poder acceder a las funcionalidades privadas del sistema, tales como la gestión de propiedades o la realización de reservas.

Este requisito garantiza el control de acceso y la correcta identificación del rol de cada usuario dentro del sistema.

RF2. Alta de propiedades por parte de propietarios

Los propietarios autenticados pueden **dar de alta propiedades**, que pueden corresponder a viviendas completas o habitaciones individuales.

Durante el alta del inmueble, el propietario define:

- Las características básicas del alojamiento.
- Las comodidades disponibles.
- La política de cancelación.
- El tipo de reserva permitido: **reserva inmediata** o **reserva bajo solicitud**.

Este requisito permite que los propietarios gestionen libremente su oferta de alojamientos dentro del sistema.

RF3. Búsqueda de alojamientos y filtros avanzados

El sistema permite a cualquier usuario realizar **búsquedas de alojamientos**, sin necesidad de estar autenticado, utilizando criterios como:

- Destino.
- Fechas.
- Tipo de inmueble.

De forma opcional, los usuarios pueden aplicar **filtros avanzados**, entre los que se incluyen:

- Disponibilidad de reserva inmediata.
- Selección de comodidades.
- Política de cancelación.

Estos filtros se modelan como extensiones del caso de uso principal de búsqueda, incrementando la flexibilidad del sistema.

RF4. Gestión de lista de deseos

Los usuarios registrados pueden **añadir alojamientos a una lista de deseos**, permitiendo almacenar propiedades de interés para futuras consultas o reservas.

Este requisito mejora la experiencia de usuario y fomenta la reutilización del sistema.

RF5. Reserva de alojamientos y pago

Los inquilinos autenticados pueden realizar reservas de alojamientos seleccionados. El sistema distingue dos escenarios:

- **Reserva inmediata:** si el inmueble lo permite, la reserva se confirma automáticamente tras completar el pago.
- **Solicitud de reserva:** si el inmueble no permite reserva inmediata, el inquilino realiza una solicitud, completando igualmente el pago en ese momento.

El sistema admite distintos **métodos de pago**:

- Tarjeta de crédito.
- PayPal.

RF6. Confirmación, denegación y devolución

En el caso de solicitudes de reserva, el propietario recibe una notificación y puede **confirmar o denegar** la solicitud.

- Si la confirmación es positiva, el sistema notifica al inquilino y la reserva queda confirmada.
- Si la solicitud es denegada, el sistema gestiona la **devolución del importe** y notifica igualmente al inquilino.

Este requisito garantiza la correcta gestión de los flujos excepcionales del sistema.

4.2 Requisitos no funcionales

Además de los requisitos funcionales, el sistema cumple una serie de **requisitos no funcionales**, implícitos en el diseño y la implementación del proyecto.

Seguridad

- Acceso restringido a funcionalidades según autenticación y rol del usuario.
- Separación clara entre usuarios autenticados y no autenticados.

Arquitectura y Mantenibilidad

- Arquitectura en capas basada en **controladores, entidades y persistencia**.
- Uso del framework **Spring Boot**, que favorece la modularidad y el mantenimiento del sistema.
- Separación entre lógica de negocio y presentación mediante controladores y plantillas.

Usabilidad

- Interfaz web desarrollada mediante **HTML y CSS**, proporcionando una navegación clara e intuitiva.
- Uso de vistas diferenciadas según el tipo de usuario. *Calidad del código*
- Uso de pruebas automatizadas con **JUnit**.
- Ejecución de pruebas integrada en el proceso de construcción mediante **Maven**.
- Análisis de calidad y cobertura del código mediante **SonarCloud** y **JaCoCo**.

4.3 Trazabilidad entre requisitos y diseño

Los requisitos funcionales descritos se reflejan directamente en el **diagrama de casos de uso** del sistema, donde se identifican claramente: •

- Los actores principales (Usuario, Inquilino y Propietario).
- Las relaciones «include» para funcionalidades obligatorias como la autenticación.
- Las relaciones «extend» para funcionalidades opcionales como filtros avanzados o tipos de reserva.

Esta trazabilidad asegura que cada requisito planteado ha sido considerado durante el diseño y posteriormente implementado en el sistema, garantizando la coherencia entre el análisis, el diseño y la implementación.

5. Diseño y arquitectura del sistema

El sistema SprintForge se ha implementado siguiendo una arquitectura web basada en el patrón MVC (Model–View–Controller), apoyada en Spring Boot. Esta arquitectura permite separar responsabilidades y mejorar la mantenibilidad:

- La aplicación está estructurada por paquetes, destacando la **capa controller** (controladores web), la **capa entity** (modelo del dominio) y la **capa persistence** (DAOs/repositorios) con las Vistas que corresponden a plantillas HTML y CSS que generan la interfaz web para el usuario.

5.1 Controladores y responsabilidades

A nivel de controladores, el sistema queda dividido por módulos funcionales:

1. GestorUsuarios

Registro, login y logout (RF1)

Implementa el acceso al sistema:

- Registro de usuarios (inquilino o propietario) y validación de login duplicado.
- Autenticación (login) y gestión de sesión.
- Logout y cierre de sesión.

Además, lanza notificaciones asociadas a eventos de registro/login/logout.

Vistas asociadas:

- registro, registroExitoso, login, inicioInquilino, inicioPropietario.

2. GestorBusquedas

Búsqueda, filtros y flujo de reserva (RF3, RF5)

Gestiona: • Página inicial de búsqueda y

resultados.

- Filtros básicos (destino, tipo, precio máximo) y avanzados (reserva inmediata, política, fechas dentro del rango disponible).
- Entrada al flujo de reserva con formulario y confirmación.
- Validaciones clave antes de crear una reserva:
 - Fechas obligatorias ○ Fecha inicio < fecha fin
 - Reserva dentro del rango de disponibilidad del propietario
 - No solapamiento con reservas existentes

(reservaDAO.existsSolapamiento) ○ No permitir
reservar inmuebles eliminados (borrado lógico)

Genera la reserva en estado ACEPTADA (si directa) o PENDIENTE (si solicitud), congela política/precio aplicados y notifica a inquilino y propietario.

Vistas asociadas:

- busqueda, formularioReserva, reservaConfirmada.

3. GestorInmuebles

Alta y gestión de propiedades (RF2)

Controla las acciones del propietario autenticado:

- Listado de inmuebles del propietario.
- Alta (/nuevo) y edición (/editar/{id}) con controles de seguridad (solo el dueño).
- Guardado (/guardar) con inicialización de campos mínimos y persistencia de Disponibilidad.
- Eliminación:
 - Si no hay reservas activas → borrado real. ○ Si hay reservas activas → borrado lógico (marcar eliminado=true).
- Envío de notificaciones por publicación/actualización/eliminación.

Incluye además vistas informativas como detalle y resultados.

Vistas asociadas:

- gestionInmuebles, forminmueble, detalleInmueble, resultados.

4. GestorReservas

Gestión de reservas (RF5, RF6)

Gestiona la visión de reservas según rol:

- Inquilino: listado de reservas propias.
- Propietario: listado de reservas de sus inmuebles.
- Guardado de nuevas reservas (flujo alternativo al de GestorBusquedas) con validaciones de fechas, rango disponible y solapamiento.
- Confirmación/aceptación y rechazo por propietario:
 - /aceptar/{id} cambia estado a ACEPTADA
 - /rechazar/{id} cambia estado a RECHAZADA y procesa devolución (lógica básica).
- Eliminación/cancelación de reserva por inquilino o propietario (y notificación).

Vistas asociadas:

- misReservas, reservasPropietario, formReserva.

5. GestorPagos

Pagos y reembolsos (RF5, RF6)

Implementa:

- Historial de pagos del inquilino.
- Formulario de pago asociado a una reserva aceptada.
- Procesado del pago: ○ Genera referencia ○ Guarda pago
 - Marca reserva como pagada y pasa a CONFIRMADA ○ Notifica a inquilino y propietario
- Cancelación de reserva confirmada por el inquilino con cálculo de reembolso según política de cancelación del inmueble.

Vistas asociadas:

- pagos, formularioPago.

6. GestorNotificaciones

Notificaciones (RF6 soporte transversal)

Este controlador ofrece:

- Visualización y gestión de notificaciones (leídas/no leídas, limpieza).
- Endpoint auxiliar GET /notificaciones/noLeidas para contador AJAX.

Además, proporciona métodos de servicio reutilizados por otros controladores para generar notificaciones de eventos de reservas, inmuebles y pagos.

Vistas asociadas:

- notificaciones.

7. GestorDeseos

Lista de deseos (RF4) Permite

al inquilino:

- Ver lista de deseos.
- Añadir/quitar inmuebles, evitando duplicados.

Vistas asociadas:

- misDeseos.

8. GestorInicio

Navegación e inicio

5.2 Controles implementados y validaciones relevantes

Para asegurar la corrección del sistema y evitar incoherencias, se han incluido validaciones importantes en los flujos críticos:

- Reservas sin solapamiento: el sistema evita reservas en fechas ya reservadas mediante `reservaDAO.existsSolapamiento(...)`.
- Reservas dentro de disponibilidad: las fechas deben estar dentro del rango `[fechaInicio, fechaFin]` definido por el propietario.
- Seguridad básica por rol:
 - Solo propietarios pueden gestionar inmuebles.
 - Solo el propietario dueño del inmueble puede editar/eliminar ese inmueble.
 - Solo el inquilino propietario de la reserva puede cancelarla.
- Borrado lógico de inmuebles cuando existen reservas activas, evitando romper consistencia.

5.3 Modelo de dominio y persistencia (capa Entity)

El sistema modela el dominio del alquiler de viviendas mediante un conjunto de entidades JPA que representan los conceptos principales del problema (usuarios, inmuebles, reservas, pagos, notificaciones y deseos). Esta capa constituye el “Model” del patrón MVC y se ha diseñado con el objetivo de mantener la coherencia de datos, soportar los flujos funcionales del enunciado y facilitar el mantenimiento del sistema.

Usuarios y roles

La entidad base es Usuario, definida con una estrategia de herencia `SINGLE_TABLE`. A partir de ella se especializan dos tipos:

- Inquilino (`@DiscriminatorValue("INQUILINO")`)
- Propietario (`@DiscriminatorValue("PROPIETARIO")`)

Este diseño simplifica el almacenamiento en una única tabla, manteniendo diferenciación de comportamiento según el rol.

Relaciones principales:

- Un Propietario tiene una lista de Inmueble (`@OneToMany(mappedBy="propietario")`).
- Un Inquilino tiene una lista de Reserva (`@OneToMany(mappedBy="inquilino")`).

Inmuebles y disponibilidad

Inmueble representa una propiedad ofertada. Incluye atributos relevantes para búsqueda y reserva (tipo, direccion, ciudad, precioPorNoche) y un mecanismo de borrado lógico mediante el flag eliminado, evitando inconsistencias cuando existen reservas activas.

Relaciones:

- Inmueble pertenece a un Propietario (@ManyToOne).
- Inmueble tiene asociada una Disponibilidad (@OneToOne(cascade=ALL)), que modela: o rango de fechas disponible (fechaInicio, fechaFin) o reserva inmediata (directa) o política de cancelación (politicaCancelacion)

La PoliticaCancelacion se modela como enum con un método getPorcentajeReembolso() que encapsula la lógica de devolución (0%, 50%, 100%).

Esto mejora la claridad y evita “magic numbers”. **Reservas y estados**

Reserva representa el vínculo entre un Inquilino y un Inmueble en un intervalo de fechas. Incluye:

- Fechas obligatorias (fechaInicio, fechaFin)
- Estado del ciclo de vida (EstadoReserva: PENDIENTE, ACEPTADA, RECHAZADA, CONFIRMADA) • Indicador de pago (pagado)

Además, para garantizar coherencia a largo plazo, la reserva congela ciertos datos del inmueble en el momento de creación:

- precioPorNocheAplicado
- politicaCancelacionAplicada
- reservaDirectaAplicada
- precioTotal

Esto evita que cambios posteriores en el inmueble (precio/política) alteren reservas ya realizadas, mejorando la consistencia del sistema.

Relaciones:

- Reserva pertenece a un Inquilino (@ManyToOne).
- Reserva pertenece a un Inmueble (@ManyToOne).
- Reserva puede tener un Pago asociado (@OneToOne(mappedBy="reserva", cascade=ALL)).

Pagos y reembolsos

Pago representa la operación de pago y su trazabilidad:

- Método de pago (MetodoPago: TARJETA, PAYPAL)
- referencia única (generada por el sistema)
- Campos sensibles (tarjeta/cvv/email) marcados como @Transient para no persistirlos en la base de datos.

- Campos de reembolso (reembolsado, importeReembolsado, fechaReembolso), necesarios para el flujo de cancelaciones/denegaciones.

Relación:

- Un Pago está asociado a una Reserva (@OneToOne con reserva_id).

Notificaciones

Esto permite implementar el requisito de notificación en reservas, pagos y gestión de inmuebles.

Lista de deseos

Deseo modela la relación entre un inquilino y un inmueble guardado en favoritos.

Se aplica una restricción de unicidad a nivel de base de datos:

- uniqueConstraints = (inquilino_id, inmueble_id)

Esto garantiza que el mismo inmueble no se añade dos veces al listado de deseos, reforzando integridad del sistema.

5.4 Capa de persistencia (DAO) y consultas clave

La persistencia se gestiona mediante interfaces DAO basadas en Spring Data JPA (JpaRepository), lo que permite:

- Operaciones CRUD estándar sin código adicional.
- Consultas derivadas por nombre de método.
- Consultas personalizadas mediante @Query cuando se requiere lógica avanzada.

DAOs principales

- UsuarioDAO: búsqueda por credenciales (findByLogin).
- PropietarioDAO, InquilinoDAO: acceso a roles específicos.
- DeseoDAO: gestión de lista de deseos y evitación de duplicados (findByInquilinoAndInmueble).
- NotificacionDAO: listado ordenado por fecha (findByUsuarioDestinoOrderByFechaDesc).
- PagoDAO: historial de pagos por inquilino (findByReserva_Inquilino).
- DisponibilidadDAO: búsqueda por inmueble (findByInmueble).
- ReservaDAO: núcleo de consultas de consistencia del dominio.

Consultas críticas en ReservaDAO

Para garantizar que el sistema cumple los requisitos de disponibilidad y consistencia, se incluyen consultas específicas:

1. Evitar solapamiento de reservas

`existsSolapamiento(inmuebleId, inicio, fin)` comprueba si ya existe una reserva para el mismo inmueble cuyo intervalo se solapa con el solicitado y cuyo estado no sea rechazado.

Esta consulta implementa directamente la restricción “no permitir reservar fechas ya reservadas”.

2. Detección de reservas activas

`existsReservaActiva(inmuebleId, hoy)` permite determinar si un inmueble tiene reservas vigentes (no rechazadas) con fecha fin posterior o igual al día actual. Se utiliza para decidir entre borrado real y borrado lógico del inmueble.

3. Edición segura del rango de disponibilidad

`existsReservaActivaFueraDeRango(inmuebleId, hoy, nuevoInicio, nuevoFin)` evita que un propietario modifique la disponibilidad de forma que deje reservas activas fuera del nuevo rango.

Este control refuerza la coherencia de datos y evita errores funcionales.

5.5 Relación directa con los requisitos

Gracias al modelo de datos y las consultas anteriores, el sistema asegura:

- RF2: Inmueble + Disponibilidad y asociación a Propietario
- RF3 (Búsqueda + filtros): atributos en Inmueble + filtros por directa y PolíticaCancelacion en Disponibilidad + rango de fechas.
- RF4 (Deseos): entidad Deseo con restricción de unicidad.
- RF5–RF6 (Reservas, pago, confirmación/denegación y devolución):
 - Reserva con estados y pago. ○ Pago con trazabilidad y campos de reembolso.
 - Notificación para comunicación automática.